Revision — 2.2.1 Programming Techniques

OCR H446 — one-page revision summary

Must know

- **Three basic constructs: sequence** (order matters), **selection/branching** (`if` / `elseif` / `else`, `switch` / `case`), **iteration**. Count-controlled (`for`) repeats a **fixed, known** number of times; condition-controlled (`while`, `do...until`) repeats until a condition changes.
- **`while`** tests at the **start** → body may run **zero** times; **`do...until`** tests at the **end** → body always runs **at least once** (good for input validation).
- **Recursion** = a subroutine that **calls itself**. Must have a **base case** (stops recursion, no further call) and a **general/recursive case** (calls itself, moving toward the base case). A missing/unreached base case → endless calls → **stack overflow**.
- **Call stack:** each call **pushes a stack frame** holding parameters, local variables and the **return address**; frames are **popped** in reverse (**LIFO**) as calls return. Recursion uses more memory than iteration but is elegant for self-similar problems (tree traversal, factorial, quicksort).
- **Scope** = the region where a variable is visible. **Local** = declared in a subroutine, visible only there, freed on exit; **global** = declared in the main program, visible everywhere. Prefer local (fewer side effects, more modular, easier to debug); a local **shadows** a same-named global.
- **Modularity** = splitting a program into small self-contained **subroutines**, each doing one task (easier to write/test/debug, reusable, splittable across a team).
- **Procedure** returns **no value** (call is a statement); **function** returns a **single value** (usable in an expression). This return is the defining difference.
- **Parameter** = variable in the subroutine *definition*; **argument** = value supplied at the *call*. **By value** passes a **copy** → original **unchanged**; **by reference** passes the **memory address** → original **can be changed** (efficient for large data).
- **IDE features:** syntax highlighting, code completion/intellisense, auto-indentation, error diagnostics, plus debugging tools — **breakpoints**, **stepping**, **variable watch** — and a built-in translator + run-time environment.
- **OOP:** a **class** is a template; an **object** is an instance. **Encapsulation** = private data + public methods (data hiding); **inheritance** = subclass acquires a superclass's attributes/methods ("is-a"); **polymorphism** = same method call behaves differently per object type, usually via **overriding**.

Key terms

Term	Definition
Sequence	Instructions executed one after another, in order
Selection	Choosing between paths based on a condition
Count-controlled iteration	Loop repeating a fixed, known number of times (`for`)
Condition-controlled iteration	Loop repeating until a condition changes (`while`, `do...until`)
Recursion	A subroutine that calls itself, with a base case and a recursive case
Base case	The condition that stops recursion (no further recursive call)
Call stack	Stack of frames holding parameters, locals and return addresses for active calls
Stack overflow	Error from too many pushed frames (e.g. missing base case)
Scope	The region of a program where a variable can be accessed
Local variable	Accessible only within the subroutine that declares it
Global variable	Accessible throughout the whole program
Procedure	Subroutine that performs a task but returns no value
Function	Subroutine that performs a task and returns a value
By value	A copy of the value is passed; the original is unaffected
By reference	The memory address is passed; the original can be changed
Encapsulation	Bundling data + methods with private attributes accessed via public methods
Inheritance	A subclass acquiring the attributes/methods of a superclass
Polymorphism	The same method call behaving differently for different object types

Worked example / how to — by value vs by reference

Start with `x = 3` and two subroutines:

```
procedure addOneByVal(byVal n)
    n = n + 1
endprocedure

procedure addOneByRef(byRef n)
    n = n + 1
endprocedure
```

- After `addOneByVal(x)` → **x = 3**. A **copy** of the value is passed, so `n` becomes 4 inside the routine but the original `x` is **unchanged**.
- After `addOneByRef(x)` → **x = 4**. The **memory address** of `x` is passed, so incrementing `n` updates the **original** `x`.

The mark is the *consequence* (original unchanged vs original changed), not simply "a copy is made".

Exam tips

- For recursion, always say it **calls itself** AND needs a **base case**; "why it fails" = **stack overflow** from a missing/unreached base case. Trace by showing frames **pushed then popped** in **LIFO** order.
- For by value vs by reference, state the *effect on the original* — by value it is **unchanged**, by reference it **can be changed**.
- Function vs procedure: the distinguishing point is that a **function returns a value** and a procedure does not.
- Keep **encapsulation** (private data + public methods) ≠ **inheritance** (reuse parent) ≠ **polymorphism** (same call, different behaviour) clearly distinct.